

Analyzing Test Case Prioritization approaches With Performance and Speed Trade-offs in CI Workflows

Abdelrahman Nader
Faculty of computer science
MSA University, Giza, Egypt
Cairo, Egypt
abdelrahman.nader@msa.edu.eg

Marwa M. Solayman
Faculty of computer science
MSA University, Giza, Egypt
Cairo, Egypt
mmsolayman@msa.edu.eg

Abstract—The software testing process is an important activity in software development that helps ensure applications are working properly and meet quality standards. Among testing methodologies, regression testing plays a pivotal role in ensuring that recent changes do not introduce new defects in previous parts of the software. However, as software applications increase in size and complexity, executing all test cases becomes increasingly difficult and time-consuming. Test case prioritization effectively addresses this challenge by identifying and selecting the most likely test cases to fail, optimizing fault detection within the available time constraints. **This study provides a comparative analysis of three state-of-the-art test case prioritization techniques: COLEMAN (Combinatorial VOLatile Multi-Armed BANDit), Reinforced Test Case Selection (RETECS), and DeepOrder.** Each model uses a different approach to prioritize test cases: combinatorial optimization, reinforcement learning and deep learning respectively. The study compares the model's ability to detect faults earlier in the testing process using various metrics such as Normalized APFD (NAPFD) and prioritization time.

Index Terms—Regression Testing, Test Case Prioritization, Software Testing, Reinforcement Learning

I. INTRODUCTION

Software testing is a vital component of the development process, ensuring system correctness

and minimizing defects. Its significance has increased with the adoption of Agile methodologies and CI/CD workflows, which require continuous testing to keep pace with rapid code changes [1]. Regression testing plays a key role by verifying that updates do not compromise existing functionality. However, executing all test cases for every change is often infeasible in complex systems. To optimize this, classical techniques such as risk-based and history-based prioritization have been used to detect faults earlier by reordering test cases [2]. Despite their benefits, these approaches are often labor-intensive and difficult to maintain. Consequently, recent advances in machine learning and artificial intelligence have led to the development of more adaptive and intelligent test case prioritization methods [3].

In this paper, we compare three approaches to test case prioritization: COLEMAN (Combinatorial VOLatile Multi-Armed BANDit) [4], Reinforced Test Case Selection (RETECS) [5], and DeepOrder [6]. Each method takes a unique approach—COLEMAN relies on combinatorial optimization, RETECS uses reinforcement learning, and DeepOrder uses deep learning techniques. We evaluate how well these models help detect faults earlier in the testing process, aiming to improve the overall efficiency of regression testing.

979-8-3315-0186-0/25/31.00©2025IEEE

II. BACKGROUND AND PROBLEM DEFINITION

Continuous Integration (CI) has become a core part of modern software development, promoting rapid code integration, frequent builds, and automated testing. While CI accelerates development cycles and improves collaboration, it also introduces new challenges in the form of maintaining testing efficiency and coverage under time constraints. As every commit triggers automated test executions and test suites grow larger and more complex, this makes exhaustive regression testing both costly and impractical in fast-paced environments. [7] To address these challenges, researchers have increasingly adopted Deep Learning (DL) and Reinforcement Learning (RL), reflecting a growing trend in recent studies toward intelligent, data-driven prioritization strategies. By learning patterns from historical test executions, DL and RL models optimize test case selection, making sure that the most fault-prone test cases are found earlier in the testing pipeline.

III. RELATED WORK

The aim of this study is to compare the performance of the recent works in the literature pertaining to deep learning and reinforcement learning that aim to optimize the process of test case prioritization (TCP).

J. Lima et al. [4] proposed COLEMAN, a TCP approach which models test cases like arms in a multi-armed bandit (MAB) setting. It handles two challenges: the combinatorial nature of selecting a collection of test cases per cycle, and volatility, where the test suite changes over time. COLEMAN applies a chosen MAB policy each CI cycle to prioritize test cases within a time or resource budget, executes them, gathers feedback, and updates rewards through a credit assignment procedure. Additionally, it is model-free so it doesn't need initial info about its environments and supports online learning leaning more into its model-free nature.

Another study by Spieker et al. [5] introduced RETECS, which applies reinforcement learning (RL) to test case prioritization using two types of agents: a Tableau representation-based agent and an artificial neural network (ANN). This study will

focus on the ANN approach as it has proved to be superior according to tests conducted in previous work [8]. RETECS is also model-free and supports online learning, making it suitable for evolving CI environments. RETECS uses policy-based decision-making to guide its choices. Based on these decisions and the test execution verdicts, it calculates a reward for the agent, which helps improve its performance over time.

The last approach discussed is the model proposed by Sharif et al. [6], DeepOrder, a deep neural network model that aims to tackle the TCP problem. DeepOrder uses a supervised deep learning approach, employing a multi-layer neural network to prioritize test cases. It processes historical execution logs from Continuous Integration (CI) cycles, applying feature extraction and data augmentation to handle imbalanced data, particularly underrepresented failed test cases. By focusing on failures, the model improves training, resulting in better prioritization.

IV. METHODOLOGY

This portion of the study will outline the experimental setup and procedures used to conduct a comparative analysis of the COLEMAN, RETECS, and DeepOrder test case prioritization models, focusing on their effectiveness in fault detection and their potential integration with real-world applications.

A. Overall Workflow

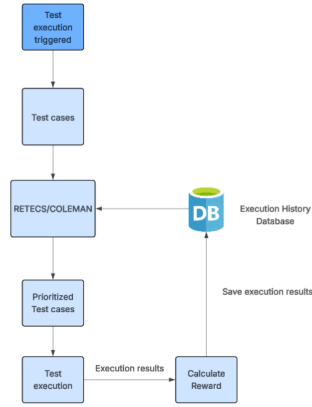
There are two proposed systems, one for each model type. Starting with the workflow utilizing reinforcement learning, it will go through several steps as follows:

- Testing execution is triggered
- Test cases are retrieved and preprocessed into a structured format compatible with the input requirements of each model.
- Test cases are prioritized using the chosen policy.
- Test execution is conducted with the prioritized test set and returns the verdict.
- The prioritization is evaluated using the test execution results and the model is rewarded accordingly.

- The execution results are saved in a database for future use.

figure 1 illustrates the workflow:

Fig. 1. System workflow utilizing RETECS/COLEMAN

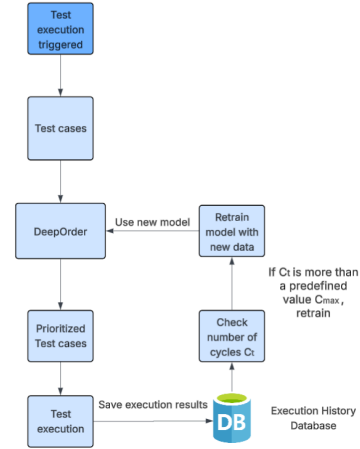


The second proposed workflow, designed for deep learning-based models, follows similar steps but differs in the reward/update phase, undergoing periodic retraining instead. The workflow will proceed like this:

- Test execution is triggered
- Test cases are retrieved and preprocessed into a structured format compatible with the input requirements of each model
- Test cases are prioritized using the DeepOrder model
- Test execution is conducted using the prioritized test set
- The execution results are saved in a database
- The retrain pipeline checks C_t , representing the number of cycles passed since the last retraining. If C_t is less than C_{max} , model retraining will commence

figure 2 outlines the whole process, with an emphasis on the retraining mechanism

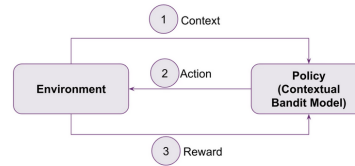
Fig. 2. System workflow utilizing DeepOrder



B. Comparative Architecture Summary

The three models, COLEMAN, RETECS, and DeepOrder, each adopt distinct architectural approaches to test case prioritization.

Fig. 3. Multi-armed bandit architecture [9]

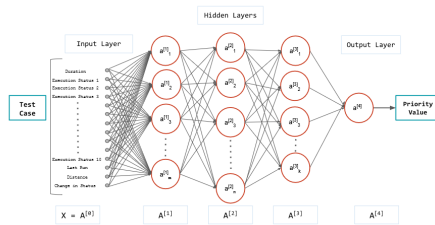


- **COLEMAN** COLEMAN utilizes a Multi-Armed Bandit (MAB) architecture, as illustrated in Figure 3. It processes system context—primarily test cases and historical execution data—and applies a user-defined policy to guide prioritization decisions. This study focuses on the **Fitness-Rate-Rank Multi-Armed Bandit (FRRMAB)** policy, a UCB-based approach that enhances selection using a rank-based quality estimator. FRRMAB calculates a Fitness Improvement Rate over a sliding window of recent outcomes to prioritize test cases with higher potential for failure detection, while accounting for recency. The feedback mechanism relies on reward functions to adapt future prioritization. Two

main reward functions are employed: Reward Based on Failures (RN-Fail), which measures the number of failures in the prioritized set, and Time-Ranked Reward (TimeRank), which penalizes non-failing test cases that precede failing ones, promoting earlier detection of failures [4].

- **RETECS** employs a reinforcement learning approach with a Q-learning agent, similar in structure to COLEMAN. This study specifically focuses on RETECS's use of an artificial neural network (ANN) as the learning agent. While COLEMAN relies on discrete action selection, RETECS uses ANNs to estimate continuous priority values based on system state inputs. It also introduces exploration through controlled noise, governed by an exploration hyperparameter that allows deviation from the learned policy. Additionally, RETECS uses experience replay, storing past interactions for periodic sampling, this enhances robustness by enabling the network to generalize from prior experiences through iterative optimization stochastic gradient descent. [5].
- **DeepOrder** (Figure 4) uses a multi-layer neural network for test case prioritization. The input layer processes 14 features, including ExecutionStatus and Duration. The network has three hidden layers with 10, 20, and 15 neurons, using the Mish activation function to address vanishing gradients and improve learning in complex, non-linear tasks [10]. The output layer produces a scalar representing test case priority. Mean Squared Error (MSE)

Fig. 4. DeepOrder model architecture [6]



is used as the loss function for performance evaluation.

C. Datasets

Fig. 5. Datasets used for experiments, [11]

Data Set	Test Cases	CI cycles	Verdicts	Failed
ABB Paint Control	114	352	25,594	19.36%
ABB IOF/ROL	2,086	320	30,319	28.43%

Figure 5 illustrates the datasets used to evaluate the models; these datasets have been utilized in many previous works [3,4,5] due to their availability .These systems differ in key characteristics like: the number of faults, the number of test cases, and the number of commits. By using these datasets, we aim to assess the generalization capacity of the approaches under different conditions and ensure a fair evaluation across various datasets. IOF/ROL, in particular, is used due to its high volatility, offering a challenging environment. Paint Control, on the other hand, is chosen for its more stable nature, making it a suitable contrast to IOF/ROL. The combination of these two datasets allows for a comprehensive evaluation, ensuring that the models are capable of handling both highly volatile and relatively stable test case scenarios.

D. Hyperparameters

The hyper parameters used in the evaluations were the best parameters illustrated in their respective papers , which are as follows:

TABLE I
COLEMAN CONFIGURATION

Hyperparameter/Policy	Value
Policy	FRRMAB
Reward Functions	RNfail, timerank
Exploration Coefficient(C) for RNfail	0.3
Exploration Coefficient(C) for timerank	0.5
Sliding window size(W)	100
Scheduling time ratio	0.5

Tables I–III summarize the configurations: COLEMAN uses FRRMAB with RNfail and timerank; DeepOrder applies an ANN with replay memory and history length of 4; and RETECS employs a

TABLE II
DEEPOrDER CONFIGURATION

Hyperparameter/Policy	Value
Agent	ANN
History Length	4
Hidden Nodes	12
Replay Memory	10000
Replay Batch Size	1000
Scheduling Time Ratio	0.5

TABLE III
RETECS CONFIGURATION

Hyperparameter	Value
Weight Initialization	Xavier (Default)
Activation Function	Mish [10] (Default)
Hidden Layer 1	10 neurons
Hidden Layer 2	20 neurons
Hidden Layer 3	15 neurons
Epochs	1000
Loss Function	Mean Squared Error
Optimizer	Adam
Learning Rate	0.001
Total Parameters	631
Trainable Parameters	631
Non-Trainable Parameters	0

three-layer network with Xavier initialization, Mish activation, and Adam optimizer.

we adopted these values directly from the original sources to enable a fair comparison under their optimal configurations, facilitating a fair and meaningful comparison of their performance.

E. Runtime and Environment

All experiments were conducted on a system with an Intel Core i5-12400F processor (6 Cores, 12 Threads, Up to 4.4GHz), an NVIDIA RTX 3050 graphics card with 8GB VRAM, 16GB of RAM, and running on a Windows 11 installation.

F. Evaluation Metrics

The first evaluation metric used in this study is Normalized Average Percentage of faults Detected (NAPFD). **Normalized Average Percentage of faults Detected (NAPFD)**: calculates the mean proportion of faults identified by the prioritized test cases. It was first proposed by [12]. It is calculated as:

$$\text{NAPFD}(T'_t) = p - \frac{\sum_{i=1}^n \text{rank}(T'_i)}{m \times n} + \frac{p}{2n}$$

Where:

- T'_t : prioritized test set.

- p : recall, which quantifies how many test cases that can detect faults are actually included .
- n : number of test cases in T' .
- $\text{rank}(T'_i)$: position of T'_i in T' , and if T'_i did not reveal a fault, the value of $T'_i = 0$.
- m : number of faults detected by the test cases.

TABLE IV
COMPARISON OF TEST CASE PRIORITIZATION TECHNIQUES
BASED ON NAPFD

Method	Reward Function	Paint Control Dataset NAPFD	IOF/ROL Dataset NAPFD
DeepOrder	—	0.5101	0.4293
COLEMAN	TimeRank	0.9148	0.4792
	FailureBased	0.9239	0.4838
RETECS	TimeRank	0.8748	0.4092
	FailureBased	0.8948	0.4238

V. RESULTS

The NAPFD results of the evaluation, can be seen in Table IV. The comparison reveals significant variations in test case prioritization effectiveness, measured by NAPFD, depending on the technique, dataset, and reward function used. COLEMAN consistently emerged as the top performer, achieving the highest NAPFD scores on both the Paint Control and the more challenging IOF/ROL datasets. COLEMAN scored a 0.9148 and 0.9239 using timerank and RNfail respectively on the Paint Control dataset, and scored a 0.47 and 0.48 on the IOF/ROL dataset. RETECS also showed strong performance on Paint Control averaging about 0.88 using both reward functions but was less effective on IOF/ROL, particularly with the TimeRank reward scoring the lowest (0.4092) out of the three models. DeepOrder presented a contrasting profile, lagging significantly on Paint Control scoring 0.5101 but performed competitively on IOF/ROL reaching 0.4292, which puts it close to RETECS's FailureBased result. Overall, the Paint Control dataset allowed for much earlier fault detection across methods compared to IOF/ROL. For the RL-based methods, the specific reward function choice had minimal impact on COLEMAN's NAPFD but provided a slight edge to RETECS when using the FailureBased reward.

TABLE V
COMPARISON OF PRIORITIZATION TIMES (IN SECONDS)

Method	Reward Policy / Variant	Paint Control Time (s)	IOF/ROL Time (s)
DeepOrder	—	0.0015	0.0018
COLEMAN	TimeRank	0.0256	0.0278
	FailureBased	0.0256	0.0278
RETECS	TimeRank	0.0028	0.05364
	FailureBased	0.0020	0.0034

The second metric used is prioritization time. Table V shows the prioritization time needed for each approach. DeepOrder demonstrates exceptional speed, requiring 0.0015s for Paint Control, 0.0018s for IOF/ROL to order the test suite. COLEMAN exhibits consistent and moderate overhead, taking approximately 0.026s to 0.028s regardless of the dataset or reward function configuration. RETECS presents significant variability; it is very fast with the FailureBased reward 0.0020s-0.0034s, but incurs a substantially higher time cost 0.05364s on the IOF/ROL dataset when using the TimeRank reward.

VI. CONCLUSION AND FUTURE WORK

This study highlights significant differences in the effectiveness and efficiency of various prioritization techniques, including MAB-based, reinforcement learning-based, and deep learning-based approaches. COLEMAN consistently achieved the highest NAPFD scores, especially on the Paint Control dataset, indicating strong and reliable fault detection. RETECS offered competitive performance on Paint Control with low overhead but struggled on the IOF/ROL dataset, particularly with the TimeRank reward. DeepOrder, while the fastest approach overall, showed limited effectiveness on Paint Control but performed comparably to RETECS on IOF/ROL. These results suggest that while COLEMAN is the most effective overall, RETECS provides a practical balance between performance and speed, and DeepOrder is best suited for scenarios where speed is the primary concern.

Future work could explore hybrid models that combine these strengths and assess scalability on larger, more varied datasets for real-world use.

Researchers could also explore the integration of these techniques into automated testing pipelines and extending their application to other types of testing such as integration and system testing.

REFERENCES

- [1] Y. Zhao, A. Serebrenik, Y. Zhou, V. Filkov, and B. Vasilescu, "The impact of continuous integration on other software development practices: A large-scale empirical study," *IEEE Xplore*, Oct. 01, 2017. <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&number=8115619>
- [2] R. Mukherjee and K. S. Patnaik, "A survey on different approaches for software test case prioritization," *Journal of King Saud University - Computer and Information Sciences*, Oct. 2018, doi: <https://doi.org/10.1016/j.jksuci.2018.09.005>.
- [3] Khatibsyarbini, Muhammad, et al. "Trend application of machine learning in test case prioritization: A review on techniques." *IEEE Access* 9 (2021): 166262-166282.
- [4] J. A. do Prado Lima and S. R. Vergilio, "A Multi-Armed Bandit Approach for Test Case Prioritization in Continuous Integration Environments," *IEEE Transactions on Software Engineering*, pp. 1–1, 2020, doi: <https://doi.org/10.1109/tse.2020.2992428>.
- [5] Spieker, Helge, Gotlieb, Arnaud, Marijan, Dusica and Mossige, Morten. (2017). Reinforcement Learning for Automatic Test Case Prioritization and Selection in Continuous Integration. 10.1145/3092703.3092709.
- [6] Sharif, Aizaz, Marijan, Dusica and Liaaen, Marius. (2021). DeepOrder: Deep Learning for Test Case Prioritization in Continuous Integration Testing. 10.48550/arXiv.2110.07443.
- [7] H. Do, "Chapter Three - Recent Advances in Regression Testing Techniques," *ScienceDirect*, Jan. 01, 2016. <https://www.sciencedirect.com/science/article/abs/pii/S0065245816300286>
- [8] Antonio do Prado Lima, Jackson and Vergilio, Silvia. (2023). An Evaluation of Ranking-to-Learn Approaches for Test Case Prioritization in Continuous Integration. *Journal of Software Engineering Research and Development*. 10.5753/jserd.2023.2142.
- [9] G. Fei, "Contextual Bandit for Marketing Treatment Optimization," *About Us*, Oct. 07, 2021. <https://www.aboutwayfair.com/careers/tech-blog/contextual-bandit-for-marketing-treatment-optimization>.
- [10] Misra, Diganta. (2019). Mish: A Self Regularized Non-Monotonic Neural Activation Function. 10.48550/arXiv.1908.08681.
- [11] Bitbucket. (n.d.). Bitbucket. <https://bitbucket.org/HelgeS/atcs-data/src/master/>
- [12] Xiao Qu, Myra B. Cohen, and Katherine M. Woolf. 2007. Combinatorial interaction regression testing: A study of test case generation and prioritization. In *IEEE International Conference on Software Maintenance*, 2007 (ICSM). IEEE, 255–264.